

UNIVERSIDADE SÃO JUDAS TADEU (USJT)
TEORIA DA COMPUTAÇÃO E COMPILADORES

Breno Henrique Ruiz dos Santos - 823131791

Maria Eduarda Medeiro Porto - 824144948

Matheus Alves Santana - 824144952

**ANALISADORES LÉXICOS: FUNDAMENTOS, TÉCNICAS E
APLICAÇÕES NA COMPILAÇÃO E INTERPRETAÇÃO**

Pesquisa Acadêmica

SÃO PAULO
2026

Breno Henrique Ruiz dos Santos – 823131791

Maria Eduarda Medeiro Porto - 824144948

Matheus Alves Santana - 824144952

ANALISADORES LÉXICOS: FUNDAMENTOS, TÉCNICAS E APLICAÇÕES NA COMPILAÇÃO E INTERPRETAÇÃO

Pesquisa Acadêmica apresentada ao UC
Teoria da Computação e Compiladores,
como parte dos requisitos necessários à
conclusão da A3.

Orientador: Milkes Alvarenga

SÃO PAULO

2026

RESUMO

Este trabalho apresenta um estudo aprofundado sobre os processos fundamentais de construção de compiladores, com foco central na fase de análise léxica. Inicialmente, estabelece-se a distinção entre os paradigmas de compilação e interpretação, situando a análise léxica como a etapa inicial do front-end de um tradutor. O estudo descreve o funcionamento do analisador léxico (scanner), detalhando a transformação de fluxos de caracteres em unidades lógicas denominadas tokens.

A fundamentação teórica baseia-se na aplicação de Gramáticas Regulares e Expressões Regulares, as quais são operacionalizadas por meio de Autômatos Finitos Determinísticos (AFD) e Não-Determinísticos (AFND), utilizando algoritmos clássicos como a Construção de Thompson e a Construção de Subconjuntos. Adicionalmente, discute-se o papel vital da Tabela de Símbolos como estrutura de dados central para o armazenamento de metadados e gestão de escopo, explorando a eficiência das Tabelas de Dispersão (Hash Tables).

Por fim, abordam-se aspectos práticos da engenharia de compiladores, como o tratamento de erros léxicos, e técnicas de otimização através da bufferização de entrada.

Palavras-chave: Compiladores. Análise Léxica. Autômatos Finitos. Tabela de Símbolos. Linguagens Formais.

Sumário

2	COMPILAÇÃO X INTERPRETAÇÃO	6
2.1	Compilação.....	6
2.2	Interpretação	6
2.3	A Análise Léxica no Processo	6
3	ANÁLISE LÉXICA	7
3.1	O Processo de Compilação	7
	Análise	7
3.2	Como funciona	8
3.3	Exemplo.....	9
3.4	Contexto no pipeline de compilação	9
3.5	Síntese	9
4	ANALISADOR LÉXICO	10
5	MECANISMOS ENVOLVIDOS NA ANÁLISE LÉXICA	11
5.1	Fundamentação Formal: Expressões Regulares e Autômatos	11
5.2	Expressões Regulares.....	11
5.3	Autômatos Finitos	12
5.4	Representação de Autômatos Finitos	13
5.5	Equivalência entre Expressões Regulares e Autômatos Finitos	13
5.6	Autômatos Finitos na Análise Léxica	14
6	PROCESSO DE CONVERSÃO E ALGORITMOS	15
7	TABELA DE SÍMBOLOS	16
7.1	Atributos e Metadados	16
7.2	Gerenciamento de Escopo e Bloco	16
8	TRATAMENTO DE ERROS LÉXICOS	17
9	ANALISADOR SINTÁTICO.....	18
10	AUTÔMATOS DE PILHA	18
	REFERÊNCIAS BIBLIOGRÁFICAS.....	19

1 INTRODUÇÃO

O desenvolvimento de software moderno assenta na capacidade de traduzir intenções humanas, expressas em linguagens de programação de alto nível, em instruções executáveis pelo hardware. Este processo de tradução é realizado por sistemas complexos denominados compiladores ou interpretadores. Enquanto os interpretadores executam o código em tempo real, os compiladores realizam uma tradução integral e prévia, otimizando o desempenho e a segurança do software final.

A arquitetura de um compilador é tradicionalmente dividida em duas grandes etapas: a análise (front-end) e a síntese (back-end). A etapa de análise é responsável por decompor o código-fonte, verificar a sua validade e extrair o seu significado estrutural. Dentro deste pipeline, a Análise Léxica surge como a porta de entrada fundamental, atuando como o primeiro filtro que transforma um fluxo contínuo de caracteres em unidades lógicas elementares, conhecidas como tokens. O sucesso desta fase inicial depende de mecanismos formais rigorosos, baseados na teoria das linguagens regulares e dos autômatos finitos. Além da identificação de padrões, a eficiência de um compilador está intrinsecamente ligada à gestão de dados através da Tabela de Símbolos, uma estrutura central que armazena metadados essenciais sobre identificadores, garantindo a consistência do programa ao longo das fases subsequentes de análise sintática e semântica.

Este trabalho propõe-se a explorar os fundamentos técnicos e teóricos que regem a análise léxica. Serão abordados desde os algoritmos de conversão de expressões regulares em autômatos determinísticos até às estratégias avançadas de tratamento de erros e bufferização. O objetivo é demonstrar como estes mecanismos convergem para criar sistemas de tradução robustos, capazes de lidar com a complexidade das linguagens de programação contemporâneas.

2 COMPILAÇÃO X INTERPRETAÇÃO

2.1 Compilação

Compilação é o processo de tradução integral do código-fonte, escrito em uma linguagem de alto nível, para uma linguagem de baixo nível, como código de máquina ou binário, que pode ser executado diretamente pelo processador. Durante esse processo, o compilador analisa o programa, identifica possíveis erros e gera um arquivo executável independente, que não necessita do código-fonte original para ser executado posteriormente.

2.2 Interpretação

Interpretação é o processo em que um programa denominado interpretador lê e executa as instruções do código-fonte em tempo real. Diferentemente da compilação, não há a geração prévia de um arquivo executável definitivo.

O interpretador traduz e executa o código conforme ele é processado, exigindo que o código-fonte, ou uma representação intermediária dele, esteja presente durante toda a execução do programa.

2.3 A Análise Léxica no Processo

A diferença da análise léxica entre linguagens compiladas e interpretadas não está em seu funcionamento interno, mas na forma como ela é utilizada dentro do processo de execução. Em ambos os casos, o analisador léxico realiza a tarefa de ler o código-fonte e transformá-lo em uma sequência de tokens. O que muda é o momento em que essa análise acontece, a frequência com que ela é executada e a reutilização de seus resultados.

Em linguagens compiladas, a análise léxica ocorre como uma etapa inicial do processo de compilação. O código-fonte é analisado uma única vez, gerando tokens que serão utilizados pelas fases seguintes, como análise sintática, análise semântica e geração de código executável. Após a compilação, o analisador léxico não participa mais da execução do programa. Já em linguagens interpretadas, a análise léxica ocorre durante o carregamento ou durante a própria execução do programa, dependendo da implementação do interpretador. Nesse contexto, o analisador léxico pode ser utilizado diversas vezes ao longo da execução, e seus resultados podem ser descartados ou convertidos em representações intermediárias, como bytecode ou estruturas internas.

Portanto, enquanto na compilação a análise léxica é utilizada como uma etapa de preparação única, focada em desempenho e otimização prévia, na interpretação ela se torna uma parte ativa do ambiente de execução, contribuindo para a flexibilidade e portabilidade do software.

3 ANÁLISE LÉXICA

3.1 O Processo de Compilação

3.1.1 Linguagens de Programação

As linguagens de programação de alto nível permitem ao programador dar instruções a uma máquina de maneira mais simples e prática do que diretamente na linguagem de máquina. Porém os processadores não entendam programas escritos em linguagens de programação de alto nível, é aí que entra o processo de compilação. As linguagens, dos mais diversos tipos e paradigmas de programação são analisadas, em busca de erros, e informações para que em seguida seja gerado um programa em linguagem de máquina, que é o objetivo do processo. A compilação de um programa é dividida em duas etapas: Análise e Síntese.

Análise

Na etapa de análise, o texto do programa é examinado, e informações sobre ele são extraídas e utilizadas no processo, e a fim de se preencher as estruturas de dados necessárias à etapa de síntese. Nesta etapa, a maioria dos erros que poderiam estar presentes no programa são detectados, sendo possível abortar o processo de compilação, ou partir para um modo de recuperação de erros onde se tenta continuar a compilação para que se possa informar o maior número de erros simultaneamente ao usuário.

Esta etapa é subdividida em três fases de análise: Léxica, Sintática e Semântica, cada uma delas trata de um aspecto do programa fonte.

- **Análise Léxica:** os aspectos léxicos do programa são tratados nesta fase. Aqui entra em cena o analisador léxico, também conhecido como scanner. Ele lê o texto do programa e o divide em tokens, que são os símbolos básicos de uma linguagem de programação. Eles representam palavras reservadas, identificadores, literais numéricos e de texto, operadores e pontuações. Comentários no texto do programa geralmente são ignorados pelo analisador.
- **Análise Sintática:** etapa realizada pelo analisador sintático, ou parser. Este analisador trabalha lendo os tokens gerados pelo analisador léxico e os agrupando de acordo com a estrutura sintática da linguagem. O resultado final é chamado de árvore de derivação.

- **Análise Semântica:** aspectos semânticos são tratados aqui. Estes aspectos levam em consideração o sentido do programa. É através da semântica que são extraídas as informações que possibilitam a posterior geração de código. Os aspectos semânticos não são facilmente especificáveis. Os principais mecanismos formais para a especificação dos aspectos semânticos são as gramáticas de atributos, as semânticas denotacionais e as semânticas de ações. Porém sua complexidade os torna inviáveis na prática, e acaba-se partindo para mecanismos semiformais, ou até mesmo informais. Uma implementação muito comum é a utilização de ações semânticas, que são inseridas no meio da especificação sintática sob a forma de sub-rotinas que são invocadas quando o parser as atinge. Durante a etapa de análise, é montada a tabela de símbolos, que mantém informações sobre todos os símbolos do programa, e é utilizada também na geração de código.

3.2 Como funciona

- **Leitura do código-fonte:** O analisador léxico (também chamado de lexer ou scanner) percorre o texto do programa de entrada.
- **Identificação de tokens:** Sequências de caracteres são agrupadas em categorias como:
 - Palavras-chave: if, else, while, return, etc.
 - Identificadores: nomes de variáveis e funções (ex: total, calcular)
 - Literais: números (42, 3.14), strings ("hello")
 - Operadores: +, -, *, ==, !=
 - Delimitadores: (,), {, }, ;
- **Remoção de elementos irrelevantes:** Espaços em branco, quebras de linha e comentários são descartados (não geram tokens).
- **Geração da sequência de tokens:** A saída é uma lista de tokens que será consumida pela próxima fase — a Análise Sintática (parsing).

3.3 Exemplo

Dado o trecho de código:

```
int x = 10;
```

O analisador léxico produziria os seguintes tokens:

Tabela 1 – Exemplo de análise léxica

Token	Tipo
int	Palavra-chave
x	Identificador
=	Operador
10	Literal Inteiro
;	Delimitador

Fonte: Os autores (2026).

3.4 Contexto no pipeline de compilação

Código-Fonte -> Análise-Léxica -> Análise-Sintática -> Análise-Semântica -> Geração de código

A análise léxica simplifica o trabalho das fases seguintes ao transformar um fluxo contínuo de caracteres em uma sequência estruturada de tokens, facilitando a construção da árvore sintática pelo parser.

3.5 Síntese

Na etapa de síntese é que é gerado o programa objeto. Nesta etapa também ocorre o processo de otimização, que elimina partes inúteis e redundantes que são geradas durante a geração intermediária de código, e mesmo aquelas introduzidas pelo programador.

4 ANALISADOR LÉXICO

O analisador léxico faz a verificação do programa quanto aos aspectos léxicos. Estes aspectos são definidos por Gramáticas Regulares (Tipo 3), ou outro mecanismo equivalente, como expressões regulares e autômatos finitos. Analisadores léxicos são implementados com base em Autômatos Finitos, que são o mecanismo formal de reconhecimento de linguagens regulares.

O analisador léxico opera em um ciclo contínuo:

1. Lê caracteres do código-fonte um a um (ou em buffer)
2. Acumula caracteres enquanto eles formam parte de um token válido
3. Reconhece o token quando encontra um caractere que não pertence mais ao padrão atual (ex: um espaço após uma sequência de letras)
4. Classifica o token (palavra-chave, identificador, número, operador, etc.)
5. Emite o token para o parser e volta ao passo 1 Internamente, cada padrão de token é definido por uma expressão regular.

Por exemplo:

- identificador → ***[a-zA-Z_][a-zA-Z0-9_]****
- número_inteiro → ***[0-9]+***
- operador_soma → ***\+***

5 MECANISMOS ENVOLVIDOS NA ANÁLISE LÉXICA

A análise léxica é fundamentalmente um processo de reconhecimento de padrões, onde o fluxo de caracteres de entrada é transformado em unidades lógicas denominadas tokens. Para que este processo ocorra de forma sistemática e eficiente, utilizam-se formalismos matemáticos que garantem a corretude do reconhecimento.

5.1 Fundamentação Formal: Expressões Regulares e Autômatos

O comportamento de um analisador léxico é regido por Gramáticas Regulares (Tipo 3 na Hierarquia de Chomsky). Na prática, os padrões de cada token são especificados via Expressões Regulares (ER), que oferecem uma notação concisa para conjuntos de cadeias de caracteres. Entretanto, para que o computador execute esse reconhecimento, as ERs devem ser convertidas em um mecanismo operacional: os Autômatos Finitos (AF). Um AF consiste em um conjunto de estados e transições acionadas por símbolos de entrada.

5.2 Expressões Regulares

Seja A um alfabeto. As expressões regulares sobre o alfabeto A e os conjuntos (linguagens) que elas denotam são definidas como:

1. ϕ é uma expressão que denota o conjunto vazio.
2. ϵ é uma expressão regular que denota o conjunto $\{\epsilon\}$.
3. Para cada símbolo a em A , a é uma expressão regular que denota o conjunto $\{a\}$.
4. Se r e s são expressões regulares que denotam os conjuntos R e S , respectivamente, então $(r|s)$, (rs) , e (r^*) são expressões regulares que denotam os conjuntos $R \cup S$, RS e R^* , respectivamente.

Na prática, quando se escreve expressões regulares, pode-se omitir muitos parênteses se for assumido que o símbolo $*$ tem precedência maior que a concatenação e $|$, e que a concatenação tem precedência maior que $|$.

Ex: ab^* , $((abc)|(cba))$, $aaa(a|a)aaa$.

5.3 Autômatos Finitos

Um autômato finito trabalha lendo uma string de símbolos e alterando seu estado interno de acordo com um controle. Se ao terminar a string o estado atual for um estado considerado final, o autômato aceita a entrada, senão rejeita.

Formalmente um autômato é uma Quíntupla: $F = (K, \Sigma, \delta, q_0, F)$, onde:

- K – conjunto de estados do autômato;
- Σ – o alfabeto de entrada do autômato;
- δ – função de $K \times \Sigma \rightarrow K$, que controla as transições entre os estados;
- q_0 – estado inicial do autômato;
- F – conjunto de estados finais.

O funcionamento de um autômato finito segue o seguinte algoritmo:

1. $T \leftarrow q_0$
2. Se a entrada possui algum símbolo ainda, então
 - a. $A \leftarrow$ próximo símbolo da entrada
3. Senão
 - a. Se T é elemento de F então Aceita
 - b. Senão Rejeita
4. Se (T, A) é elemento do domínio de δ então
 - a. $T \leftarrow \delta(T, A)$
 - b. Vá para 2
5. Senão
 - a. Rejeita.

5.4 Representação de Autômatos Finitos

Autômatos finitos podem ser representados por diagramas de transições, que são um grafo dirigido e rotulado, os vértices representam os estados e as arestas as transições. O estado inicial possui uma seta que o indica, e os finais possuem círculos duplos.

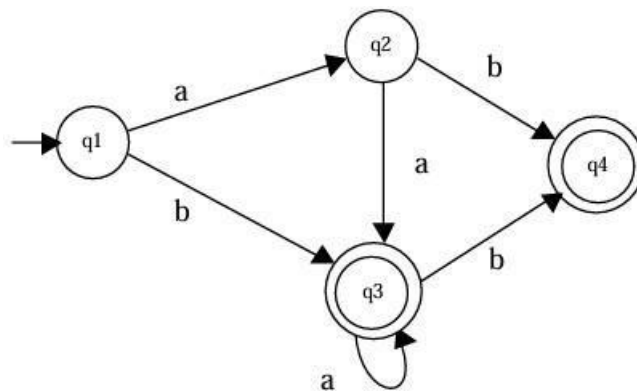
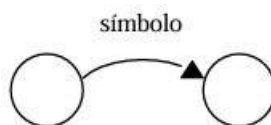


Figura 1 - representação Autômatos Finitos

5.5 Equivalência entre Expressões Regulares e Autômatos Finitos

Pode-se facilmente obter um autômato finito não-determinístico a partir de uma expressão regular. A ideia principal é gerar uma série de pequenos autômatos, que são unidos via ϵ -transições. Cada um dos símbolos da expressão dá origem a um autômato da seguinte forma: símbolo associado a transição entre estados.



5.6 Autômatos Finitos na Análise Léxica

A implementação utilizada em geral é utilizar a tabela de transições do autômato finito e um algoritmo que faz a análise léxica percorrendo a sentença e consultando a tabela. Uma outra possibilidade ainda é a de se programar o autômato diretamente, sabendo-se o estado atual, e o próximo símbolo da sentença, pode-se avançar para outro estado, prosseguindo-se dessa forma até acabar a sentença, ou não ser possível fazer a transição de estados.

O analisador léxico deve identificar os diferentes tokens da linguagem, e devolver ao analisador sintático um código para este token. É preciso então algum mecanismo para que o autômato finito faça isto. Geralmente é feito com que o autômato tenha vários pontos de saída, um para cada token diferente.

Para as palavras reservadas da linguagem também é utilizada uma técnica diferente: sempre que um token de identificador é detectado, faz-se uma busca numa tabela por essa cadeia, se for encontrada, é retornado o token equivalente, senão, é retornado o token de identificador mesmo.

Na prática, o que se utiliza são geradores de analisadores léxicos, que são ferramentas que produzem automaticamente analisadores léxicos, normalmente a partir de uma especificação baseada em gramáticas regulares, expressões regulares ou autômatos finitos.

6 PROCESSO DE CONVERSÃO E ALGORITMOS

A construção automatizada de um analisador léxico segue um pipeline algorítmico rigoroso:

- **Construção de Thompson:** Transforma cada expressão regular em um Autômato Finito Não-Determinístico (AFND). AFNDs podem ter múltiplas transições para o mesmo símbolo ou transições espontâneas ϵ -transições, o que dificulta a implementação direta em código.
- **Algoritmo de Construção de Subconjuntos:** Converte o AFND em um Autômato Finito Determinístico (AFD) equivalente. No AFD, para cada estado e símbolo de entrada, existe exatamente uma transição, permitindo uma execução em tempo linear $O(n)$ em relação ao tamanho da entrada.
- **Minimização de Estados:** Opcionalmente, reduz o número de estados do AFD ao mínimo necessário, otimizando o uso de memória e a velocidade de processamento.

7 TABELA DE SÍMBOLOS

A Tabela de Símbolos (TS) é uma estrutura de dados dinâmica e central que atua como o "cérebro" do compilador durante todas as suas fases. Sua função primordial é armazenar informações sobre os identificadores encontrados no código-fonte, como variáveis, funções, constantes e tipos.

7.1 Atributos e Metadados

Para cada entrada na TS, o compilador armazena atributos essenciais para as fases de análise semântica e geração de código:

- Nome (Lexema): A cadeia de caracteres original.
- Tipo: Inteiro, real, booleano, ou tipos complexos.
- Escopo: O nível de visibilidade (global ou local) para evitar colisões de nomes.
- Endereço/Deslocamento: Localização de memória onde o valor será armazenado em tempo de execução.

7.2 Gerenciamento de Escopo e Bloco

Linguagens modernas permitem que o mesmo nome seja usado em contextos diferentes (ex: variáveis locais em funções distintas). A Tabela de Símbolos gerencia isso através de estruturas de pilha de tabelas ou encadeamento, onde cada novo bloco abre um novo nível de visibilidade, garantindo que o compilador sempre acesse a declaração correta para o contexto atual.

8 TRATAMENTO DE ERROS LÉXICOS

Um analisador léxico robusto não deve apenas reconhecer tokens válidos, mas também gerenciar sequências de caracteres que não pertencem à linguagem. O erro léxico ocorre quando o scanner não consegue associar o prefixo do código restante a nenhuma expressão regular definida.

- Estratégias de Recuperação:
- Modo Pânico: O analisador descarta caracteres sucessivos até encontrar um caractere delimitador (como espaço ou ponto e vírgula) para tentar retomar a análise do próximo token.
- Reparação de Localidade: O analisador tenta transformar a entrada incorreta em algo válido (ex: deletar um caractere estranho, inserir um caractere faltante ou trocar um caractere por outro).
- Exemplos Comuns: Identificadores que começam com números (em linguagens que não permitem), caracteres ilegais (como @ em linguagens onde ele não é operador) ou strings constantes que não foram fechadas antes do fim da linha.

9 ANALISADOR SINTÁTICO

O analisador sintático faz a verificação do programa quanto aos aspectos sintáticos. Estes aspectos são definidos por Gramáticas Livres de Contexto (Tipo 2). O mecanismo formal para o reconhecimento desta classe de gramáticas é o Autômato de Pilha.

Existem duas estratégias para a implementação de analisadores sintáticos, estas estratégias são: a Top-Down, ou Descendente, e a Bottom-Up, ou Ascendente.

10 AUTÔMATOS DE PILHA

O autômato de pilha é uma extensão do autômato finito onde existe uma pilha para manter o controle extra que é necessário para o gerenciamento de linguagens livres de contexto.

Um autômato de pilha é uma Séptupla $P = (K, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, onde:

- K - conjunto de estados do autômato;
- Σ - alfabeto de entrada; Γ - alfabeto da pilha; δ - função de $(K \times (\Sigma \cup \{\epsilon\}) \times \Gamma) \rightarrow (K \times \Gamma^*)$ que controla as transições de estado e a pilha;
- q_0 - estado inicial do autômato;
- Z_0 - símbolo inicial da pilha;
- F – Conjunto de símbolos finais.

Os movimentos do autômato de pilha se dão através da função de mapeamento. A partir do estado atual, do próximo símbolo da entrada, e do topo da pilha, se obtém o próximo estado e os símbolos que serão empilhados no lugar do anterior. O símbolo da entrada é consumido no processo. No caso de a transição possuir ϵ como símbolo da entrada, esta transição pode ocorrer qualquer que seja o símbolo da entrada, e ele não será consumido.

A notação gráfica dos autômatos de pilha é parecida com a notação dos autômatos finitos. O que muda são os rótulos das transições. Ao invés de apenas indicar o símbolo da entrada, é necessário indicar também o símbolo que deve estar no topo da pilha e a transformação a se fazer na pilha.

REFERÊNCIAS BIBLIOGRÁFICAS

AHO, A. V.; SETHI, R.; ULLMAN, J. D. **Compilers: Principles, Techniques and Tools**. Menlo Park: Addison-Wesley, 1986.

ALCANTARA, F. **Tabela de Símbolos em Compiladores Modernos – Linguagens Formais e Autômatos**. Disponível em:

<<https://frankalcantara.com/lf/11tabelaSimbolos.html>>. Acesso em: 5 maio 2026.

COSTA, R. H. P. et al. **Compiladores**. Londrina: Editora e Distribuidora Educacional, 2018.

FEDOZZI, R. **Compiladores**. Londrina: Editora e Distribuidora Educacional, 2018.

GESSER, C. E. **GALS: Gerador de Analisadores Léxicos e Sintáticos**. Trabalho de Conclusão de Curso (Bacharelado em Ciências da Computação) – Universidade Federal de Santa Catarina (UFSC), Florianópolis, 2002.

JOHANN, M. **INF01033 - Compiladores B: Gramáticas Livres de Contexto**. Porto Alegre: Universidade Federal do Rio Grande do Sul (UFRGS). Disponível em:

<<https://www.inf.ufrgs.br/~johann/comp/aula04.glc.pdf>>. Acesso em: 5 maio 2026.

MACÊDO, F. **Autômatos e Análise Léxica**. Rio de Janeiro: Universidade Federal do Rio de Janeiro (UFRJ). Disponível em: <<https://ic.ufrj.br/~fabiom/comp/03Automatos.pdf>>. Acesso em: 5 maio 2026.

NASCIMENTO, J. S. et al. **Desenvolvimento de um Analisador Léxico Sobre a Gramática de uma Determinada Linguagem**. Cruz Alta: Universidade de Cruz Alta (UNICRUZ), 2015.

PRICE, A. M. A.; TOSCANI, S. S. **Implementação de Linguagens de Programação: Compiladores**. Porto Alegre: Sagra-Luzzatto, 2001. (Série Livros Didáticos, n. 9).

UNIVERSIDADE DE ÉVORA. **Expressões Regulares - Autômatos e Linguagens de Programação**. Disponível em: <https://home.uevora.pt/~fc/alp/01palavras_linguagens_expressoes_regulares/01.02-expressoes_regulares.html>.

Acesso em: 5 maio 2026.

UNIVERSIDADE FEDERAL DE PERNAMBUCO (UFPE). **Autômatos e Linguagem: Expressões Regulares**. Recife: Centro de Informática. Disponível em: <https://www.cin.ufpe.br/~lfsc/cursos/teoriadainformacao/unidade%201/cap%201_7%20%20automatos%20e%20linguagem%20expressoes%20regulares.pdf>. Acesso em: 5 maio 2026.

